

TS-FCTP (Two-Stage Fixed Charge Transportation Problem)

Seminarski rad – Tema broj 8

Mateja Stojanović (mi251107)

Osnovne informacije

- **Ime i prezime:** Mateja Stojanović
- **Broj indeksa:** mi251107
- **Tema/Zadatak:** Tema broj 8

1 Opis problema

Problem TS-FCTP (Two-Stage Fixed Charge Transportation Problem) predstavlja problem transporta u dve faze kroz mrežu sa tri nivoa čvorova:

- proizvođači (izvori) $i \in I$,
- distributivni centri $j \in J$,
- kupci $k \in K$.

Roba se transportuje u dve etape:

$$i \rightarrow j \rightarrow k.$$

Za svaku vezu u mreži postoje:

- **varijabilni trošak** po jedinici protoka,
- **fiksni trošak** koji se plaća ukoliko se veza koristi (tj. ukoliko se šalje neka količina robe).

Cilj je minimizacija ukupnog troška transporta uz poštovanje ograničenja kapaciteta izvora i ispunjenje potražnje kupaca.

2 Matematička formulacija problema

2.1 Skupovi i parametri

- I – skup proizvođača (izvora), $|I| = I$,
- J – skup distributivnih centara, $|J| = J$,
- K – skup kupaca, $|K| = K$,
- s_i – raspoloživa ponuda izvora i ,
- d_k – potražnja kupca k ,
- b_{ij} – varijabilni trošak transporta od izvora i do centra j ,
- f_{ij} – fiksni trošak aktiviranja veze (i, j) ,
- c_{jk} – varijabilni trošak transporta od centra j do kupca k ,
- g_{jk} – fiksni trošak aktiviranja veze (j, k) .

2.2 Promenljive

- $x_{ij} \geq 0$ – količina koja se transportuje od izvora i do centra j ,
- $y_{jk} \geq 0$ – količina koja se transportuje od centra j do kupca k ,
- $z_{ij} \in \{0, 1\}$ – indikator da li se koristi veza (i, j) ,
- $w_{jk} \in \{0, 1\}$ – indikator da li se koristi veza (j, k) .

2.3 Funkcija cilja

Minimizuje se suma varijabilnih i fiksnih troškova u obe faze:

$$\min \sum_{i \in I} \sum_{j \in J} (b_{ij}x_{ij} + f_{ij}z_{ij}) + \sum_{j \in J} \sum_{k \in K} (c_{jk}y_{jk} + g_{jk}w_{jk}).$$

2.4 Ograničenja

Ograničenje ponude izvora

$$\sum_{j \in J} x_{ij} \leq s_i, \quad \forall i \in I.$$

Zadovoljenje potražnje kupaca

$$\sum_{j \in J} y_{jk} = d_k, \quad \forall k \in K.$$

Balans protoka u distributivnim centrima Ukupni ulaz u centar mora biti jednak ukupnom izlazu:

$$\sum_{i \in I} x_{ij} - \sum_{k \in K} y_{jk} = 0, \quad \forall j \in J.$$

Aktivacija fiksnih troškova (Big-M povezivanje) Ako se veza koristi, binarna promenljiva mora biti 1:

$$\begin{aligned}x_{ij} &\leq s_i z_{ij}, & \forall i \in I, \forall j \in J, \\y_{jk} &\leq d_k w_{jk}, & \forall j \in J, \forall k \in K.\end{aligned}$$

3 Značenje funkcije cilja i uslova

- **Funkcija cilja** predstavlja ukupni trošak transporta:
 - $b_{ij}x_{ij}$ i $c_{jk}y_{jk}$ su varijabilni troškovi proporcionalni količini robe,
 - $f_{ij}z_{ij}$ i $g_{jk}w_{jk}$ su fiksni troškovi koji se plaćaju jednom ako se veza koristi.
- **Ograničenja ponude** obezbeđuju da nijedan izvor ne šalje više od raspoložive količine.
- **Ograničenja potražnje** garantuju da se zahtevi svih kupaca tačno zadovolje.
- **Balans u centrima** znači da distributivni centri ne mogu praviti zalihe niti stvarati robu, već samo prosleđuju protok.
- **Big-M ograničenja** povezuju kontinualne tokove sa binarnim promenljivama, čime se modeluje fiksni trošak aktivacije veze.

4 Instance korišćene u eksperimentu

4.1 Podela instanci (male, srednje, velike, baseline)

Instance su generisane u tri grupe na osnovu dimenzija (I, J, K) :

Male instance (20 instanci)

Instanca	Dimenzije (I, J, K)
small_1	(4,8,16)
small_2	(4,8,20)
small_3	(4,10,20)
small_4	(6,12,24)
small_5	(6,12,30)
small_6	(6,15,30)
small_7	(8,16,32)
small_8	(8,16,40)
small_9	(8,20,40)
small_10	(10,20,40)
small_11	(10,20,50)
small_12	(10,25,50)
small_13	(12,24,48)
small_14	(12,24,60)
small_15	(12,30,60)
small_16	(14,28,56)
small_17	(14,28,70)
small_18	(16,32,64)
small_19	(16,32,80)
small_20	(18,36,72)

Srednje instance (20 instanci)

Instanca	Dimenzije (I, J, K)
medium_1	(20,40,80)
medium_2	(20,40,100)
medium_3	(20,50,100)
medium_4	(25,50,100)
medium_5	(25,50,125)
medium_6	(25,60,120)
medium_7	(30,60,120)
medium_8	(30,60,150)
medium_9	(30,75,150)
medium_10	(35,70,140)
medium_11	(35,70,175)
medium_12	(40,80,160)
medium_13	(40,80,200)
medium_14	(45,90,180)
medium_15	(45,90,225)
medium_16	(50,100,200)
medium_17	(50,100,250)
medium_18	(55,110,220)
medium_19	(55,110,275)
medium_20	(60,120,240)

Velike instance (20 instanci)

Instanca	Dimenzije (I, J, K)
large_1	(70,140,280)
large_2	(70,140,350)
large_3	(80,160,320)
large_4	(80,160,400)
large_5	(90,180,360)
large_6	(90,180,450)
large_7	(100,200,400)
large_8	(100,200,500)
large_9	(110,220,440)
large_10	(110,220,550)
large_11	(120,240,480)
large_12	(120,240,600)
large_13	(130,260,520)
large_14	(130,260,650)
large_15	(140,280,560)
large_16	(140,280,700)
large_17	(150,300,600)
large_18	(150,300,750)
large_19	(160,320,640)
large_20	(160,320,800)

4.2 Baseline instance

Pored tri osnovne grupe instanci (male, srednje i velike), generisana je i posebna grupa *baseline* instanci. Ove instance služe kao referentni skup za poređenje performansi solvera, jer imaju umerene dimenzije.

Baseline grupa sadrži 4 instance sledećih dimenzija (I, J, K):

Instanca	Dimenzije (I, J, K)
baseline_1	(10,20,40)
baseline_2	(15,25,50)
baseline_3	(20,30,60)
baseline_4	(25,40,80)

4.3 Primer jedne manje instance

Format instance fajla je:

- prva linija: $I \ J \ K$
- druga linija: ponude s_i
- treća linija: potražnje d_k
- zatim matrice troškova: $b_{ij}, f_{ij}, c_{jk}, g_{jk}$

Primer (struktura) male instance `small_1.txt` dimenzija (4, 8, 16):

```

4 8 16
80 80 80 80
... (16 vrednosti potraznje koje zbirno daju 320)
... (matrica b_ij dim 4x8)
... (matrica f_ij dim 4x8)
... (matrica c_jk dim 8x16)
... (matrica g_jk dim 8x16)

```

5 Generisanje instanci

Instance su generisane programom `generator.cpp`. Koordinate čvorova se nasumično biraju iz opsega $[-400, 400]$, a rastojanja se računaju Euklidskom metrikom i zaokružuju na dole.

Ponude su fiksne:

$$s_i = 80 \quad \forall i \in I,$$

dok su potražnje inicijalno generisane uniformno u opsegu $[10, 30]$, a zatim se koriguju tako da važi:

$$\sum_{k \in K} d_k = \sum_{i \in I} s_i.$$

Varijabilni troškovi su proporcionalni rastojanju, dok se fiksni troškovi dobijaju množenjem rastojanja slučajnim faktorom:

- za prvu fazu faktor $r_f \in [10, 15]$,
- za drugu fazu faktor $r_g \in [5, 10]$.

Kod generatora instanci (`generator.cpp`)

```

1 #include <fstream>
2 #include <vector>
3 #include <cmath>
4 #include <random>
5 #include <string>
6
7 struct Node {
8     int x, y;
9 };
10
11 int dist(const Node& a, const Node& b) {
12     return (int)std::floor(
13         std::sqrt((a.x - b.x)*(a.x - b.x) +
14                 (a.y - b.y)*(a.y - b.y))
15     );
16 }
17
18 void generate_group(const std::string& prefix,
19                    const std::vector<std::vector<int>>& dims,
20                    std::mt19937& rng) {
21

```

```

22 std::uniform_int_distribution<int> coord(-400,400);
23 std::uniform_int_distribution<int> rf(10,15);
24 std::uniform_int_distribution<int> rg(5,10);
25 std::uniform_int_distribution<int> dem(10,30);
26
27 for (int t = 0; t < (int)dims.size(); t++) {
28     int I = dims[t][0];
29     int J = dims[t][1];
30     int K = dims[t][2];
31
32     std::vector<Node> P(I), D(J), C(K);
33     for (auto& x : P) x = {coord(rng), coord(rng)};
34     for (auto& x : D) x = {coord(rng), coord(rng)};
35     for (auto& x : C) x = {coord(rng), coord(rng)};
36
37     std::vector<int> s(I, 80);
38     std::vector<int> d(K);
39     int supply = I * 80, demand = 0;
40
41     for (int k = 0; k < K; k++) {
42         d[k] = dem(rng);
43         demand += d[k];
44     }
45
46     for (int k = 0; demand != supply; k = (k+1)%K) {
47         if (demand > supply && d[k] > 1) {
48             d[k]--; demand--;
49         } else if (demand < supply) {
50             d[k]++; demand++;
51         }
52     }
53
54     std::ofstream out("generated_instances/" + prefix +
55                     "_" + std::to_string(t+1) + ".txt");
56
57     out << I << " " << J << " " << K << "\n";
58
59     for (int i = 0; i < I; i++) out << s[i] << " ";
60     out << "\n";
61
62     for (int k = 0; k < K; k++) out << d[k] << " ";
63     out << "\n";
64
65     for (int i = 0; i < I; i++) {
66         for (int j = 0; j < J; j++)
67             out << dist(P[i], D[j]) << " ";
68         out << "\n";
69     }
70
71     for (int i = 0; i < I; i++) {
72         int r = rf(rng);

```

```

73         for (int j = 0; j < J; j++)
74             out << dist(P[i], D[j]) * r << "␣";
75         out << "\n";
76     }
77
78     for (int j = 0; j < J; j++) {
79         for (int k = 0; k < K; k++)
80             out << dist(D[j], C[k]) << "␣";
81         out << "\n";
82     }
83
84     for (int j = 0; j < J; j++) {
85         int r = rg(rng);
86         for (int k = 0; k < K; k++)
87             out << dist(D[j], C[k]) * r << "␣";
88         out << "\n";
89     }
90
91     out.close();
92 }
93 }
94
95 int main() {
96
97     std::mt19937 rng(1389);
98
99     std::vector<std::vector<int>> small = {
100         {4,8,16}, {4,8,20}, {4,10,20}, {6,12,24}, {6,12,30},
101         {6,15,30}, {8,16,32}, {8,16,40}, {8,20,40}, {10,20,40},
102         {10,20,50}, {10,25,50}, {12,24,48}, {12,24,60}, {12,30,60},
103         {14,28,56}, {14,28,70}, {16,32,64}, {16,32,80}, {18,36,72}
104     };
105
106     std::vector<std::vector<int>> medium = {
107         {20,40,80}, {20,40,100}, {20,50,100}, {25,50,100},
108         {25,50,125},
109         {25,60,120}, {30,60,120}, {30,60,150}, {30,75,150},
110         {35,70,140},
111         {35,70,175}, {40,80,160}, {40,80,200}, {45,90,180},
112         {45,90,225},
113         {50,100,200}, {50,100,250}, {55,110,220}, {55,110,275},
114         {60,120,240}
115     };
116
117     std::vector<std::vector<int>> large = {
118         {70,140,280}, {70,140,350}, {80,160,320}, {80,160,400},
119         {90,180,360},
120         {90,180,450}, {100,200,400}, {100,200,500}, {110,220,440},
121         {110,220,550},
122         {120,240,480}, {120,240,600}, {130,260,520}, {130,260,650},
123         {140,280,560},

```



```

117         {140,280,700}, {150,300,600}, {150,300,750}, {160,320,640},
118             {160,320,800}
119     };
120     std::vector<std::vector<int>> tiny = {
121         {3, 5, 10}, {4, 6, 12}, {5, 8, 15}
122     };
123     std::vector<std::vector<int>> baseline = {
124         {10, 20, 40}, {15, 25, 50}, {20, 30, 60}, {25, 40, 80}
125     };
126
127     generate_group("tiny", tiny, rng);
128     generate_group("baseline", baseline, rng);
129     generate_group("small", small, rng);
130     generate_group("medium", medium, rng);
131     generate_group("large", large, rng);
132
133     return 0;
134 }
135

```

6 CPLEX implementacija

Model je implementiran u C++ korišćenjem IBM ILOG CPLEX Concert Technology. Vreme izvršavanja je ograničeno na 2h (7200s), a broj niti na 1 radi konzistentnosti.

Kod rešenja (CPLEX C++)

```

1  #include <fstream>
2  #include <ilcplex/ilocplex.h>
3  #include <iostream>
4  #include <iomanip>
5
6  ILOSTLBEGIN
7
8  int main(int argc, char* argv[]) {
9      if (argc < 2) {
10         std::cerr << "Usage: ./solve_tsfc_instance.txt\n";
11         return 1;
12     }
13
14     std::ifstream in(argv[1]);
15     if (!in) {
16         std::cerr << "Cannot open file.\n";
17         return 1;
18     }
19
20     IloEnv env;
21

```

```

22     IloInt I, J, K;
23     in >> I >> J >> K;
24
25     IloNumArray s(env, I);
26     IloNumArray d(env, K);
27
28     for (IloInt i = 0; i < I; i++) in >> s[i];
29     for (IloInt k = 0; k < K; k++) in >> d[k];
30
31     IloArray<IloNumArray> b(env, I);
32     IloArray<IloNumArray> f(env, I);
33     IloArray<IloNumArray> c(env, J);
34     IloArray<IloNumArray> g(env, J);
35
36     for (IloInt i = 0; i < I; i++) {
37         b[i] = IloNumArray(env, J);
38         f[i] = IloNumArray(env, J);
39     }
40
41     for (IloInt j = 0; j < J; j++) {
42         c[j] = IloNumArray(env, K);
43         g[j] = IloNumArray(env, K);
44     }
45
46     for (IloInt i = 0; i < I; i++)
47         for (IloInt j = 0; j < J; j++) in >> b[i][j];
48
49     for (IloInt i = 0; i < I; i++)
50         for (IloInt j = 0; j < J; j++) in >> f[i][j];
51
52     for (IloInt j = 0; j < J; j++)
53         for (IloInt k = 0; k < K; k++) in >> c[j][k];
54
55     for (IloInt j = 0; j < J; j++)
56         for (IloInt k = 0; k < K; k++) in >> g[j][k];
57
58     in.close();
59
60     try {
61         IloModel model(env);
62
63         IloNumVarArray x(env, I * J, 0.0, IloInfinity, ILOFLOAT);
64         IloBoolVarArray z(env, I * J);
65         IloNumVarArray y(env, J * K, 0.0, IloInfinity, ILOFLOAT);
66         IloBoolVarArray w(env, J * K);
67
68         auto idx_x = [J](IloInt i, IloInt j) { return i * J + j; };
69         auto idx_y = [K](IloInt j, IloInt k) { return j * K + k; };
70
71         IloExpr obj(env);
72         for (IloInt i = 0; i < I; i++)

```

```

73         for (IloInt j = 0; j < J; j++)
74             obj += b[i][j] * x[idx_x(i,j)] + f[i][j] * z[idx_x(
              i,j)];
75
76     for (IloInt j = 0; j < J; j++)
77         for (IloInt k = 0; k < K; k++)
78             obj += c[j][k] * y[idx_y(j,k)] + g[j][k] * w[idx_y(
              j,k)];
79
80     model.add(IloMinimize(env, obj));
81     obj.end();
82
83     for (IloInt i = 0; i < I; i++) {
84         IloExpr expr(env);
85         for (IloInt j = 0; j < J; j++)
86             expr += x[idx_x(i,j)];
87         model.add(expr - s[i] <= 0);
88         expr.end();
89     }
90
91     for (IloInt k = 0; k < K; k++) {
92         IloExpr expr(env);
93         for (IloInt j = 0; j < J; j++)
94             expr += y[idx_y(j,k)];
95         model.add(expr - d[k] == 0);
96         expr.end();
97     }
98
99     for (IloInt j = 0; j < J; j++) {
100         IloExpr expr(env);
101         for (IloInt i = 0; i < I; i++)
102             expr += x[idx_x(i,j)];
103         for (IloInt k = 0; k < K; k++)
104             expr -= y[idx_y(j,k)];
105         model.add(expr == 0);
106         expr.end();
107     }
108
109     for (IloInt i = 0; i < I; i++)
110         for (IloInt j = 0; j < J; j++)
111             model.add(x[idx_x(i,j)] - s[i] * z[idx_x(i,j)] <=
              0);
112
113     for (IloInt j = 0; j < J; j++)
114         for (IloInt k = 0; k < K; k++)
115             model.add(y[idx_y(j,k)] - d[k] * w[idx_y(j,k)] <=
              0);
116
117     IloCplex cplex(model);
118     cplex.setParam(IloCplex::Param::TimeLimit, 7200);
119     cplex.setParam(IloCplex::Param::Threads, 1);

```

```

120     cplex.setOut(env.getNullStream());
121
122     if (cplex.solve() && cplex.isPrimalFeasible()) {
123         std::cout << std::fixed << std::setprecision(2);
124         std::cout << argv[1] << " "
125             << ((cplex.getStatus() == IloAlgorithm::
126                 Optimal) ? "OPTIMAL" : "FEASIBLE") << " "
127             << cplex.getObjValue() << " "
128             << cplex.getTime() << " "
129             << cplex.getNiterations() << " "
130             << cplex.getNnodes() << "\n";
131     }
132
133     } catch (IloException &e) {
134         std::cerr << "CPLEX Exception: " << e << std::endl;
135     }
136
137     env.end();
138     return 0;
139 }

```

7 Rezultati CPLEX-a

Vreme izvršavanja je ograničeno na 2 sata po instanci. U tabelama su prikazani:

- status (OPTIMAL ili FEASIBLE),
- vrednost ciljne funkcije,
- vreme izvršavanja (s),
- broj iteracija,
- broj čvorova.

7.1 Male instance (small)

Instanca	Status	Obj	Vreme (s)	Iteracije	Čvorovi
small_1	OPTIMAL	150568.00	1.30	494	13
small_2	OPTIMAL	156022.00	1.51	969	48
small_3	OPTIMAL	161224.00	1.92	745	60
small_4	OPTIMAL	186537.00	1.90	467	11
small_5	OPTIMAL	213785.00	2.06	754	18
small_6	OPTIMAL	188954.00	7.15	9348	583
small_7	OPTIMAL	231984.00	17.97	22618	1308
small_8	OPTIMAL	202467.00	11.79	8786	367
small_9	OPTIMAL	210740.00	25.79	28303	1551
small_10	OPTIMAL	258236.00	7.38	124646	5628

Instanca	Status	Obj	Vreme (s)	Iteracije	Čvorovi
small_11	OPTIMAL	286991.00	84.08	1168471	37922
small_12	FEASIBLE	284643.00	1818.90	5118195	119052
small_13	OPTIMAL	278044.00	43.03	100035	4193
small_14	OPTIMAL	284648.00	2509.23	7374762	207598
small_15	OPTIMAL	315417.00	1585.38	3325528	71013
small_16	FEASIBLE	349820.00	3479.76	6251954	101284
small_17	FEASIBLE	368942.00	2606.42	5470859	128331
small_18	FEASIBLE	349036.00	4359.04	8837101	171084
small_19	FEASIBLE	351345.00	4018.04	7789292	142852
small_20	FEASIBLE	400600.00	3203.13	5178550	110993

7.2 Srednje instance (medium)

U nastavku su prikazane instance za koje su rezultati dostupni:

Instanca	Status	Obj	Vreme (s)	Iteracije	Čvorovi
medium_11	FEASIBLE	551427.00	4242.68	8148431	228474
medium_12	FEASIBLE	478570.00	7194.29	9805905	250345

7.3 Baseline instance

U eksperimentu su testirane i baseline instance:

Instanca	Status	Obj	Vreme (s)	Iteracije	Čvorovi
baseline_1	OPTIMAL	223136.00	1.70	14281	404
baseline_2	OPTIMAL	317975.00	1190.98	11745067	265058
baseline_3	OPTIMAL	322093.00	3695.07	21479433	385399
baseline_4	FEASIBLE	399062.00	7189.11	1280093	31964

8 Komentar i analiza rezultata

Na osnovu dobijenih rezultata može se uočiti sledeće:

- Za manje instance CPLEX u velikom broju slučajeva pronalazi optimalno rešenje relativno brzo, ali kako dimenzije rastu, broj čvorova i iteracija značajno raste.
- Kod nekih instanci (npr. `small_12`, `small_16-small_20`) CPLEX nije uspeo da dokaže optimalnost u okviru vremenskog limita, ali je pronašao dopustivo rešenje (status FEASIBLE).
- Za srednje instance prikazane u tabeli (`medium_11` i `medium_12`), CPLEX je dostigao vremenski limit i dao samo dopustiva rešenja, što ukazuje da problem postaje veoma težak već u ovoj dimenziji.
- Vreme izvršavanja je u korelaciji sa rastom dimenzija i kompleksnošću MILP modela zbog binarnih promenljivih z_{ij} i w_{jk} , koje uvode kombinatornu eksploziju u pretrazi.

Napomena o rezultatima većih instanci

U trenutnoj verziji rada nisu prikazani rezultati za sve instance iz skupa `medium`, kao nijedna iz skupa `large`. Tokom izvršavanja eksperimenta pojavili su se tehnički problemi prilikom rešavanja pojedinih instanci: na više različitih mašina dolazilo je do prekida rada računara, dok je za instancu `medium_10` program u više pokušaja neočekivano prekidao izvršavanje bez ispisivanja rezultata. Zbog navedenog, u radu su trenutno prikazani samo rezultati za instance za koje je izvršavanje uspešno završeno. Planirano je da se preostali eksperimenti ponove na drugoj konfiguraciji, nakon čega će rezultati biti naknadno dopunjeni.

Napomena o rezultatima i pribavljanju referentnih instanci

U okviru ovog rada nije bilo moguće izvršiti poređenje dobijenih rezultata sa referentnim vrednostima iz literature, jer originalne instance korišćene u radu autora nisu bile dostupne za preuzimanje. Iako je u literaturi naveden link ka instancama, nakon višenedeljnih pokušaja pristupa i slanja upita, kao i direktnog kontaktiranja istraživača koji su sprovedi istraživanje, instance nisu pribavljene.

9 Link za kod, izvrsnu verziju i korišćene instance

Kod programa i izvršna verzija (CPLEX) dostupni su na sledećem linku:

<https://github.com/MSZ2/TS-FCTP-Matheuristics-project/tree/CPLEX>

Literatura

- [1] H. I. Calvete, C. Galé, J. A. Iranzo, P. Toth, *A matheuristic for the two-stage fixed-charge transportation problem*.